

Application Message Format:

In this Peer-to-peer system we have defined our messages that go across the peers and to the server as a Dictionary of the following format:

```
{  
'message_type': "a",  
'message_payload': "b"  
}
```

Now, there are two kinds of message transfer in this system

- Peer to Registration Server
- Peer to Peer

Peer ⇌ Registration Server

Register Peer With Registration Server:

a="Register"

b={"hostname": "x.y.w.z", "port_number": "#", "cookie_number": "n"}

Where n = -1 when the peer is registering itself for the first time

Response:

a="Register"

b={"cookie_number": "n"}

Leave the Network:

a="Leave"

b={"cookie_number": "n"}

Response:

a="Leave"

b="successfully left"

PQuery

a="PQuery"

b={"hostname": "x.y.w.z", "port_number": "#", "cookie_number": "n"}

Response:

a="PQuery"

b="response_list", where response list is the list containing dictionaries of Peers as {"hostname": "x.y.w.z", "port_number": "#"}"

Keep Alive:

```
a="KeepAlive"
b={" cookie_number":"n"}
```

Response:

```
a="KeepAlive"
b="reset of timer done"
```

Peer ⇌ Peer:**RFCQuery:**

```
a="GetRfcList"
b={"connection_details":{"hostname":"x.y.w.z","port_number":"n"},rfc="-1"}
```

Response

```
a="GetRfcList"
b={"peer":rfc_list} , where rfc_list is the list of rfc_dictionary of following type
{
  'rfc_number' : "#",
  'title': "title_of_rfc",
  'hostname' : "this is the origin host of the RFC",
  'ttl': 7200
}
```

E.g response

```
{
  "hostname of peer1":
  { 'rfc_number' : "#",
    'title': "title_of_rfc",
    'hostname' : "this is the origin host of the RFC",
    'ttl': 7200
  }
}
```

GetRFC:

```
a="GetRFC"
```

```
b={'connection_details':{'hostname':"x.y.w.z",'port_number':"n"}, 'rfc':"rfc_dictionary"}
```

Where rfc_dictionary would be of format

```
{  
  'rfc_number' : "#",  
  'title': "title_of_rfc",  
  'hostname' : "this is the origin host of the RFC",  
  'ttl': 7200  
}
```

Response:

This message would get a response as a byte stream of chunks of data.

Experimental Setup:

- We have used lxd containers installed with CentOS running on Ubuntu 16.04 to demonstrate multiple peers
- lxd containers give us file and network isolation. All the containers are attached to lxdbr0 bridge making all on same subnet with different host id's
- The RFCs are sent to the receiver at 1024 bytes per packet.

Task 1:
Centralized File Distribution

TASK 1 :Centralized File Distribution

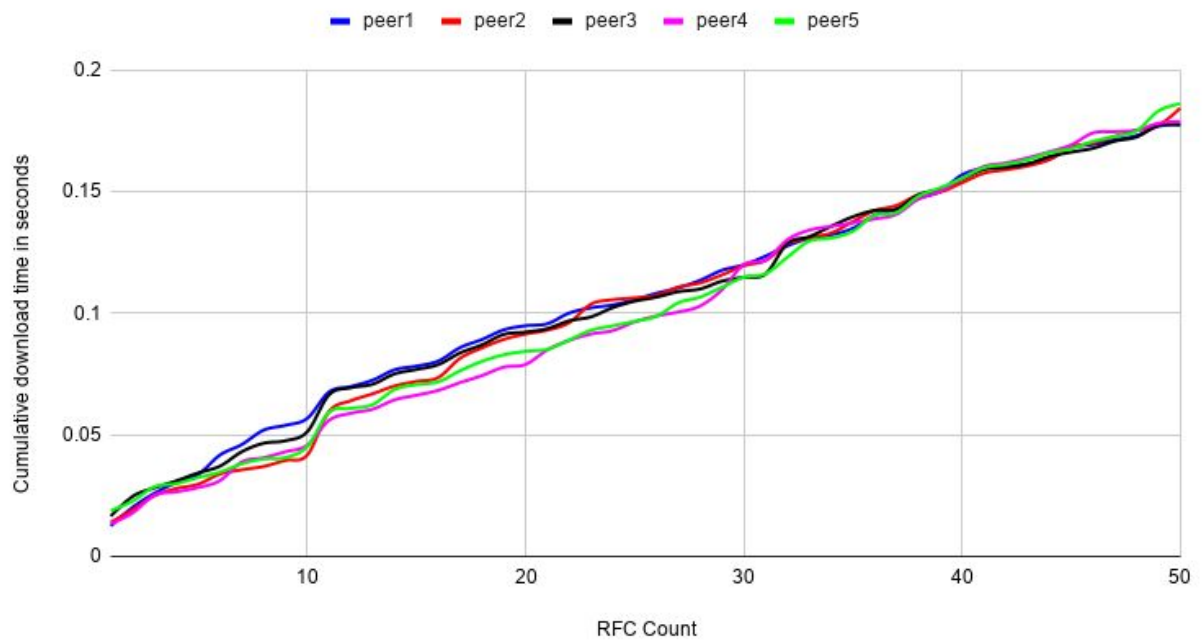


Figure 1

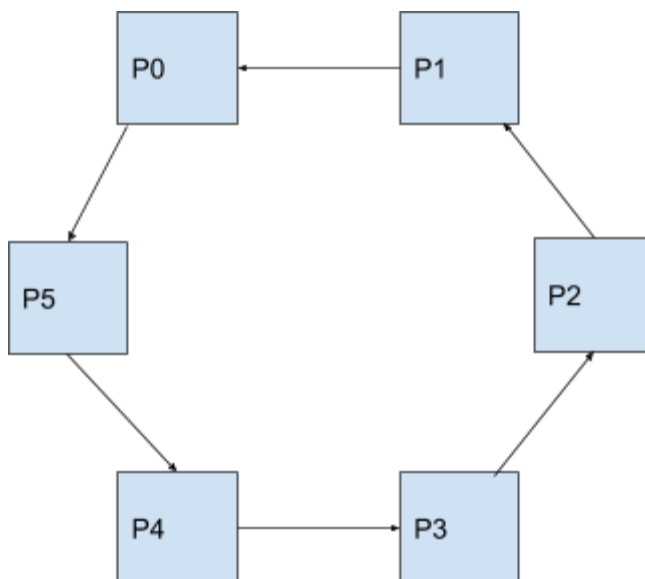
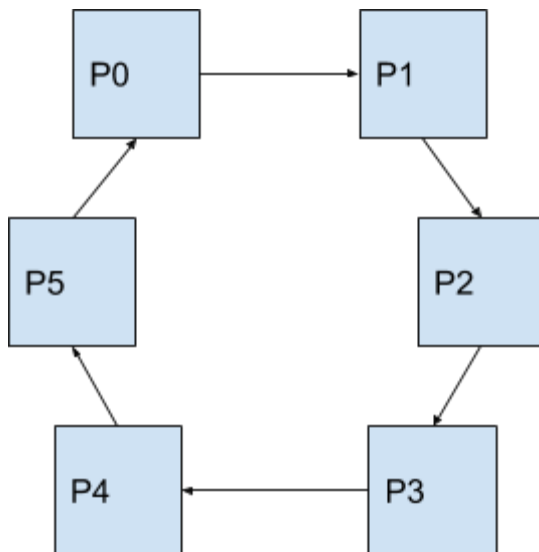
Task 2:

P2P File Distribution:

Best case scenario :

All the peers are connections in a way such that no two peers connect to the same one reducing the queuing delay at the interface of the server

Ex. $P1 \leftrightarrow P2$, $P2 \leftrightarrow P3$, $P4 \leftrightarrow P5$, $P5 \leftrightarrow P1$



TASK 2: P2P File Distribution

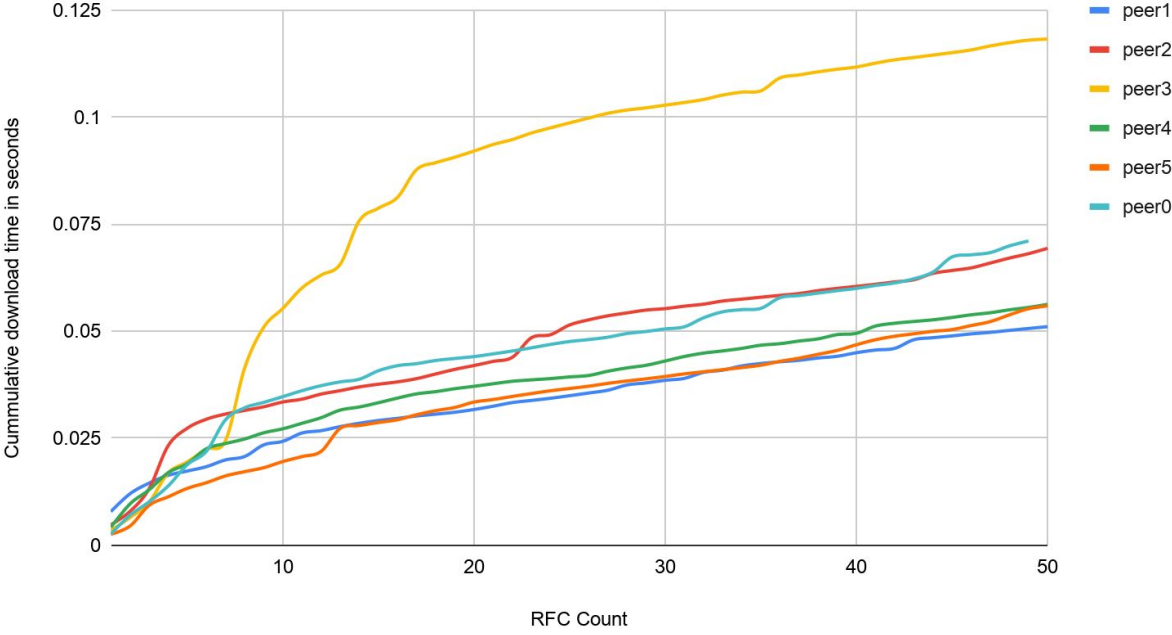
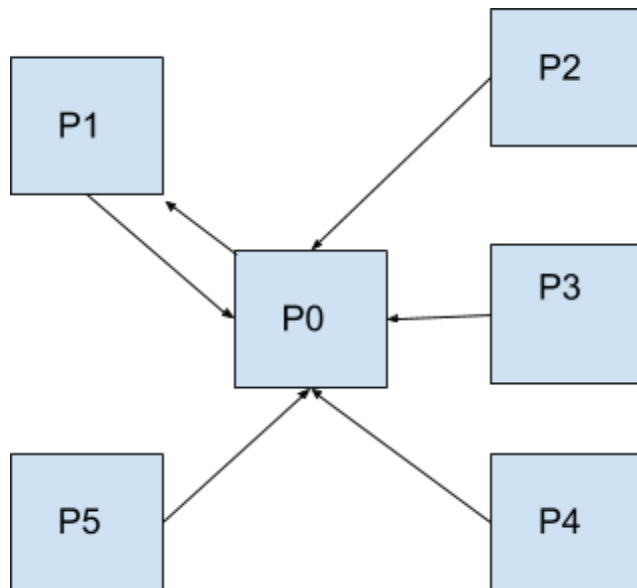


Figure 2

Worst Case scenario:

All the peers connect to a single peer at a time requesting RFC at the same time.

Ex. $P1 \leftrightarrow P0$, $P2 \leftrightarrow P0$, $P3 \leftrightarrow P0$, $P4 \leftrightarrow P0$, $P5 \leftrightarrow P0$



TASK 2: P2P File Distribution

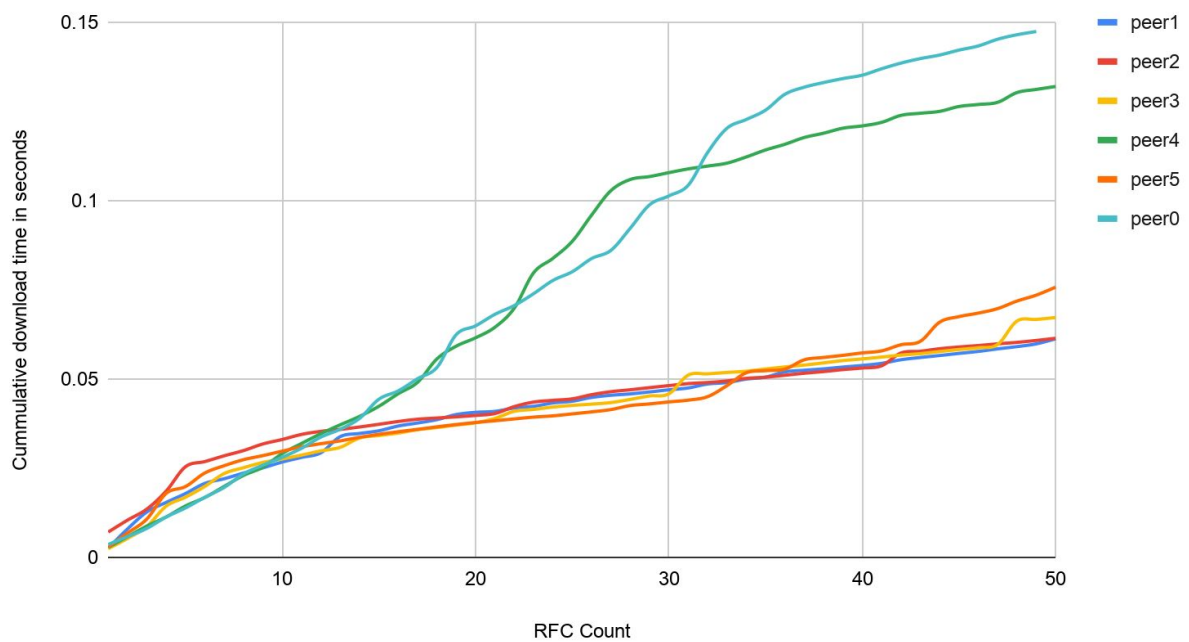


Figure 3

Differences Observed In the Graphs

As we can see in Figure 1, the plot for the centralised File distribution has a relatively higher download time distribution values compared to that of P2P File distribution as we see in Figure 2 and Figure 3, This is because in centralized file sharing all the peers connect to a single host containing all the files, and these files would experience queuing delay while being sent to the peer.

Also as we can see in Figure 2 and Figure 3 that the Download distribution time for Best case scenario is less than that in Worst case scenario, although we expect the worst case scenario to be almost similar to that of Centralized file sharing (as in this case all the peers are requesting RFC from a single peer, which in turn is requesting RFC's from one of the other peers) it is closer to the best case but higher as even in that case we have a situation like central server where all peers connect to the same at one time.

There are a few unbound cases that do not follow the trend, and it might be due to the unpredictable networks and also might be due to unsynchronized peers that try to connect.

Peer to peer network is more scalable as there would be multiple servers which can handle the client request reducing the load on the interface on any one server making the network request much faster as we can see from the provided results.
